

Experiment No. 02:

Experiment Title: Controlling the LED blink rate using potentiometer

Objectives

- Learn how to interface an analog input device (potentiometer) with an Arduino Uno.
- Read variable voltage levels using the Arduino's Analog-to-Digital Converter (ADC).
- Map analog sensor values to a time delay parameter in embedded C/C++.
- Dynamically adjust the blinking frequency of an LED using dynamic analog inputs.

Introduction

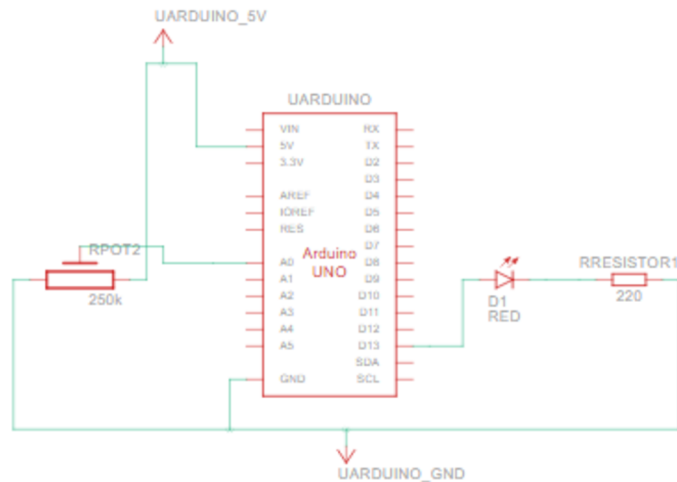
The Arduino Uno contains an onboard 10-bit Analog-to-Digital Converter (ADC) connected to analog pins A0 through A5. This feature allows the microcontroller to interpret continuous analog voltage signals ranging from 0V to 5V and map them to integer values from 0 to 1023.

- **Potentiometer Mechanics:** A potentiometer is a three-terminal variable resistor. The two outer pins connect across power (5V) and Ground (GND), while the center pin (wiper) provides a variable voltage output dependent on the mechanical knob position.
- **ADC Mapping:** As the knob rotates, the voltage on the wiper pin fluctuates between 0V and 5V. The ATmega328P ADC reads this voltage as a proportional numeric value ($0 \rightarrow 0\text{V}$, $1023 \rightarrow 5\text{V}$), which can directly serve as a variable delay interval in milliseconds for LED switching.

Apparatus Required

Component / Equipment	Quantity	Specification / Description
Arduino Uno R3	1	Microcontroller development board
LED	1	Standard 5mm Light Emitting Diode
Potentiometer	1	10kΩ Rotary Potentiometer
Resistor (220Ω)	1	Current-limiting resistor for the LED
Breadboard	1	Solderless prototyping board
Jumper Wires	As needed	Male-to-Male jumper wires

Circuit Diagram



Working Procedure

1. Drag and place the Arduino Uno, LED, 10k Ω Potentiometer, and 220 Ω Resistor onto the Tinkercad workspace.
2. Connect the cathode (short leg) of the LED directly to the Arduino's GND pin.
3. Connect the anode (long leg) of the LED to a 220 Ω resistor, and connect the resistor's second lead to Digital Pin 13.
4. Connect one outer pin of the potentiometer to the 5V power pin on the Arduino.
5. Connect the opposite outer pin of the potentiometer to the GND rail.
6. Connect the center pin (wiper) of the potentiometer to Analog Pin A0 on the Arduino.
7. Open the Tinkercad code window, select the C++ text editor, and write the program.
8. Click Start Simulation, rotate the potentiometer knob, and observe how the speed of the blinking LED alters in response.
- 9.

Code

```
void setup() {  
  // put your setup code here, to run once:  
  // designate pin 13 as an OUTPUT pin for the LED  
  pinMode (13, OUTPUT);  
  // designate pin A0 as an Analog INPUT for the potentiometer  
  pinMode (A0, INPUT);  
  Serial.begin(9600);}  
  
void loop() {  
  // put your main code here, to run repeatedly:  
  Serial.println (A0);  
  int potValue = analogRead (A0);  
  digitalWrite (13, HIGH);  
  delay (potValue);  
  digitalWrite (13, LOW);  
  delay (potValue);}
```

Discussion and Conclusion

- Discussion: The circuit operated seamlessly by pairing analog inputs with digital outputs. Rotating the potentiometer knob varied the voltage on pin A0 between 0V and 5V, which the 10-bit ADC converted into values from 0 to 1023. Passing this variable into the delay() function produced dynamic control over the blinking frequency—yielding rapid flashing near 0V and a slow 1-second pulse near 5V.
- Final Verdict: The experiment was executed successfully. The Arduino accurately captured variable analog signals from the potentiometer and utilized them to dynamically control the execution rate of the digital LED blinking cycle.